```
fact 'the tracking application tracks a user mouse click', ->
  options = {}
  tracking = new Tracking options, http
  tracking.start ->
    assert.equal tracking.total(), 0
    fred = new User serverOptions, http
    fred.visit '/page/1', ->
      fred.clickMouse ->
        assert.equal tracking.total(), 1
       tracking.stop()
```

The remainder of the test is passed as the callback to the start method of the tracking object.

Leaving the callback in the test for the `start` method works well for a while. But what if the `start` method has some external dependency? What if the `start` method performs some potentially error-prone network I/O?

10.4.2. Remove it

There are two types of functions that accept callbacks. One uses I/O and the other one doesn't. *Don't do any I/O in a unit test*. Remember the `http` double?

```
http =
  get: (options, callback) ->
    callback()
    @
  on: (event, callback) ->
    # do nothing
```

The `get` method takes a callback as an argument but it isn't asynchronous; it's invoked immediately. It's unlike a real `http` where the callback is invoked only after a network request has successfully returned. This means the `http` double has removed the dependency on the external HTTP service whenever it's used in place of the real `http`. If you pass in an `http` double as a dependency to the `tracking` object, then you're removing the external dependency from the test:

```
fact 'the tracking application tracks a user mouse click', ->
  options = {}
  http =
    get: (options, callback) ->
      callback()
      @
```